



College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID511001: Programming 2
Level 5, Credits 15
Project 1

Assessment Overview

In this **individual** assessment, you will design and develop a **Windows Forms Application** using **C#**.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Build interactive, event-driven GUI applications using pre-built components.
2. Declare and implement user-defined classes using encapsulation, inheritance and polymorphism.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Project 1	25%	Sunday 9th November at 11.59 PM	1, 2
Project 2	35%	Sunday 14th September at 11.59 PM	1, 2
Theory Examination	30%	Wednesday 8th and Thursday 9th October from 10.00 AM - 11.45 AM	1, 2
Classroom Tasks	10%	Sunday 12th October at 11.59 PM	1, 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November at 11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID511001: Programming 2**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at <https://www.op.ac.nz/about-us/governance-and-management/policies>.

Submission

You **must** submit all application files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – <https://classroom.github.com/a/Xg7esDmf>. If you do not have not one, create a **.gitignore**

and add the ignored files in this resource - <https://raw.githubusercontent.com/github/gitignore/main/VisualStudio.gitignore>.

Create a branch called **project-1**. The latest application files in the **project-1** branch will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID511001: Programming 2**.

Instructions

You will need to submit an application and documentation that meet the following requirements:

Functionality - Learning Outcome 1 (55%)

- The game needs to be driven by one **Timer** and begins when a player presses the **space bar** key.
- The ball and two paddles need to be created using the **Graphics** class.
- The ball needs to collide off the top and bottom of the screen, and paddles.
- The paddles need to move vertically but not exceed the top and bottom of the screen.
- The player one controls the left paddle via the **W** and **S** keys. The player two controls the right paddle via the **up arrow** and **down arrow** keys.
- Display the player one and player two's score using the **DrawString** method.
- When the ball collides with the left and right-hand side of the screen, one point is given to either players. The game is over when either score is 10.
- When the game is over, appropriate feedback needs to be displayed to the players.
- Double buffering to prevent the ball, paddles and scores from flickering.
- Play a sound when the ball bounces off the paddle, and top and bottom of the screen, and when a player wins or loses.
- Restart and pause/resume a game when the player presses the **R** and **P** keys respectively.
- Randomise the colour of the ball and paddles.

Code Quality and Best Practices - Learning Outcome 2 (40%)

- Naming Conventions (5%) - File, class, method, variable, constant, property names and access modifiers should be clear, descriptive and consistent across the codebase.
- Documentation and Comments (10%) - The code should be well-documented using the **XML** documentation format with comments that explain complex logic and clarify the purpose of classes, methods or complex sections.
- Code Formatting (10%) - Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections.
- Dead Code (5%) - The code should not contain unused or redundant files, classes, methods, variables, constants or properties. Keeping unnecessary code around can lead to confusion, clutter and potential bugs down the line.
- Performance and Scalability (10%) - The code should be optimised for performance, using **Object-Oriented Programming** principles, efficient algorithms and data structures to minimise bottlenecks.

Class Diagram - Learning Outcome 2 (2.5%)

- Generate a class diagram using Visual Studio's built-in tools. This diagram should include all classes, methods and properties used in the application.

Version Control - Learning Outcome 2 (2.5%)

- Your **Git commit messages** should reflect the context of each functional requirement change.

Additional Information

- You may add additional classes and methods.
- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.